The Webflow App Stack — Vite + TypeScript Monorepo, with 11ty/Vue/Svelte Islands

One monorepo, three packages: **Vite + TS** designer extension, **CF Worker** data client, **shared types** end-to-end — and on the site side, **islands** (11ty or Astro) where pages stay static and components hydrate one at a time.

For: anyone building a hybrid Webflow app (designer extension + data client) or a headless Webflow-adjacent site, who wants type safety end-to-end and zero framework runtime where it isn't earned.

Credit where due: the app-side scaffold follows the pattern in Web Bae's "Steal This Webflow App Template!" and its template repo — Svelte + TS + Vite + Cloudflare Workers in a pnpm monorepo. This doc maps that pattern onto a production stack and extends it with the site-side islands setup.

Part A — the hybrid app: one monorepo, three packages

A "hybrid" Webflow app (Webflow's term) is two programs pretending to be one: a **designer extension** running in an iframe inside the Designer, and a **data client** — your server — talking to the Webflow Data API. The monorepo makes them one project with one type system:

```
webflow-app/
├── pnpm-workspace.yaml
├── packages/
│   ├── client/          # designer extension — Vite + TypeScript + Svelte (or Vue)
│   │   ├── webflow.json          # extension manifest
│   │   ├── vite.config.ts        # production build → bundle.zip for Webflow
│   │   ├── vite-dev.config.ts    # hot-reload dev server inside the Designer
│   │   └── src/App.svelte
│   ├── common/          # THE POINT: shared types — write once, import from both sides
│   │   └── src/main.ts           # request/response contracts, entity types
│   └── server/          # data client — Cloudflare Worker on the edge
│       ├── wrangler.toml
│       ├── worker-configuration.d.ts
│       └── src/index.ts
```

Why each choice earns its place:

- **pnpm workspaces** — `packages/common` is a real dependency of both `client` and `server`. Change a contract type once; both sides fail to compile until they agree. That is the whole game: *the extension "knows" what the server will send, and the server validates what the extension asks* — type-safe end-to-end, enforced by the compiler instead of by discipline.
- **Vite + TypeScript for the extension** — instant dev server with hot reload *inside the Designer* (the template wires a dev config for exactly this), and a production build that emits the flat bundle Webflow's CLI uploads. No hand-compiled TS, no stale build folders.
- **Svelte (or Vue) for the extension UI** — the extension iframe is a genuine app surface (panels, wizards, state), which is where a component framework earns its runtime. Svelte compiles most of itself away; Vue is the same idea with a different accent. Pick one; the monorepo shape is identical.

- **Cloudflare Worker as the data client** — serverless, on the edge, `wrangler.toml` in the same repo, `worker-configuration.d.ts` typing the bindings. OAuth secrets and Data API calls live here, never in the extension.

The contract package in practice:

```
// packages/common/src/main.ts
export interface PublishRequest { siteId: string; slugs: string[] }
export interface PublishResult  { ok: boolean; published: string[]; errors?: string[] }
```

```
// packages/server/src/index.ts — the Worker validates what it receives
import type { PublishRequest, PublishResult } from "common";
```

```
// packages/client/src/lib/api.ts — the extension knows what it gets back
const res: PublishResult = await fetch(api("/publish"), { method: "POST", body: JSON.stringify(
```

Migrating an existing app (an extension that hand-compiles TS beside a standalone worker): keep both deployables exactly where they are; add `pnpm-workspace.yaml` above them, extract the request/response interfaces you're currently duplicating into `packages/common`, and point both `tsconfig`s at it. Vite adoption can follow later — the shared-types win comes first and costs an afternoon.

Part B — the site side: 11ty islands (Vue/Svelte where earned, HTML everywhere else)

The app stack above is for the *app*. The site — marketing pages, docs, storefront-adjacent surfaces — deserves the opposite default: **static HTML with zero framework runtime**, and hydration only where a component genuinely needs state. That is islands architecture, and 11ty ships it first-party:

- **Eleventy** renders the pages at build time — templates, collections, data cascade. No client runtime at all.
- **Vite** runs as the asset pipeline (via `@11ty/eleventy-plugin-vite`) — TypeScript, HMR in dev, hashed bundles in prod. One toolchain for app and site.
- `<is-land>` (11ty's official islands component) wraps each interactive component and controls *when* it hydrates:

```
<is-land on:visible>
  <pricing-configurator></pricing-configurator>
  <template data-island>
    <script type="module" src="/islands/pricing-configurator.js"></script>
  </template>
</is-land>
```

- **Vue or Svelte per island, not per page.** Each island is its own Vite entry — a Svelte component compiled to a custom element, or a Vue component mounted onto its placeholder. The page stays server-rendered; the island's JS loads `on:visible`, `on:idle`, or `on:interaction`. A page with no islands ships no framework bytes.

The Astro variant. If you'd rather the islands be first-class syntax than a component you wire, Astro is the same architecture with the pattern built in:

pages are server-rendered `.astro` files that ship **zero JS by default**, and each island declares its own hydration trigger inline — `<PricingConfigurator client:visible />` (`client:load`, `client:idle`, `client:only` as needed). It is framework-agnostic per island — Vue, Svelte, even mixed in one project — and Vite is Astro's native bundler, so the toolchain stays one thing across app and site. Choose by temperament: **11ty +** `<is-land>` keeps you closest to plain HTML with islands as progressive enhancement; **Astro** trades a framework's conventions for less wiring. The rules below apply identically to both.

Rules that keep it honest:

1. **The static render is the real content.** Every island must render meaningful HTML before hydration (SEO, AEO, and no-JS all read the same page). Hydration adds behavior, never content. The gold standard for what that static render must carry — semantics, machine-readable structure, the accessibility and machine-index scores — is the Accessibility & Machine Index spec; hold every island page to it.
2. **One design system, framework-free.** Tokens and CSS come from the shared stylesheet both platforms already consume; islands style with the same classes. No framework-specific styling systems.
3. **Framework budget per island, reviewed like a dependency.** "This card needs Svelte" is a claim to justify, not a default. Most interactivity on content pages is a `<details>`, a `<dialog>`, or thirty lines of vanilla TS in a Vite entry.
4. **Contracts still come from `packages/common`.** An island that talks to the Worker imports the same request/response types as the designer extension. One type system across app, site, and server.

The payoff, measured in what each surface ships: the Designer extension ships a framework because it *is* an app; the site ships HTML because it *is* a document; the islands ship exactly the JS their behavior requires; and the Worker types all of it.

Part C — the Turbopack risk, defined (hand this to the next Tailwind lover)

The counter-proposal is always the same, and it arrives from both directions — agencies pitching velocity and CTOs pitching consolidation: *"just build it in Next.js; Turbopack is fast."* Speed is not the question. The question is **who owns your execution order**, and it decomposes into three defined risks:

Risk 1 — order-of-execution dependency. In a document, `<script>` order is a contract: the browser executes top to bottom, and "consent loads before analytics" is guaranteed by position. Under a bundler-scheduled app, execution order is an *emergent property* of the chunk graph: code-splitting boundaries, streaming server components, Suspense resolution, prefetch heuristics, and hydration scheduling all decide what runs when. The order still *exists* — you just no longer author it. Any requirement phrased as "X must run before Y" (consent before tracking, token check before render, design tokens before paint) has moved from a guarantee to a probability.

Risk 2 — the race classes. Three recur in production and resist reproduction in dev:

- *Hydration vs. third-party*: the consent gate hydrates while an analytics SDK, injected by a different chunk, has already fired its first beacon. The gate "usually wins." Usually is not a compliance term.
- *Effect timing vs. external load*: `useEffect` chains assume an SDK global exists; whether it does depends on chunk arrival order, which differs by route, cache state, and connection.
- *Style insertion order*: Tailwind utilities plus any CSS-in-JS arrive in chunk order, not stylesheet order. Two visits can compose the cascade differently —

the specificity flip that "can't be reproduced locally," because the dev server and the production bundler do not order chunks the same way. Dev/prod parity is precisely what a dev-speed-optimized bundler trades away.

Risk 3 — code-load requirements. Regulated surfaces carry load-order *requirements*, not preferences: the consent baseline must execute **first, on every page, exactly once, deterministically** — because an auditor's question is "prove nothing ran before the gate," and "our bundler usually schedules it first" is not an answer. Meeting that inside the framework means fighting it (`beforeInteractive` confined to the root layout, still bundler-transformed, still upstream of hydration races). Meeting it in a document is a one-line `<script>` tag at the top of `<head>`.

This requirement has a name and an enforcement arm: **Google Consent Mode v2**, mandatory for EEA traffic since March 2024. Four signals — `ad_storage`, `analytics_storage`, `ad_user_data`, `ad_personalization` — must be **defaulted to *denied* before any Google tag executes**, then updated when the user chooses. Get the order wrong and the failure is silent and commercial, not a console warning: Google drops the unconsented events from measurement, audience and remarketing lists stop filling, and Smart Bidding loses the signal it bids on — the ad budget keeps spending against a shrinking data base. CMv2 is a load-order requirement enforced by a third party's revenue pipeline, which makes Risk 1 a finance problem.

And the stakes moved because the KPI moved: marketing was built on the page view; the funnel now pays for the consented login. When revenue measurement keys off consented, identified sessions rather than raw traffic, an execution order you cannot guarantee is a revenue leak — every race the consent gate loses is a session your bidding never sees.

The acceptance test, one sentence: *can you guarantee — not observe, guarantee — that the consent gate executes before any third-party code, on every route, in production, and show the test that enforces it?* If the answer is

yes, the stack is fine. If the answer is a re-run of the demo, the risk is defined above. Islands pass the test by construction: the document loads in author order, and the framework only ever runs inside a boundary you drew.

Companion reading: Webflow App Requirements Checklist — the Marketplace compliance side of the same build · Architecture · the video and template this scaffold follows: Web Bae — Steal This Webflow App Template! / webflow-app-monorepo.
